
SVMs and QSVM

Release -

Álvaro Sánchez-Paniagua

May 15, 2025

Contents

Add your content using reStructuredText syntax. See the [reStructuredText](#) documentation for details.

1.1 Submodules

1.2 Analysis.plots module

`Analysis.plots.plot_accuracy_diff_table(df_comp: DataFrame) → None`

Muestra una tabla con la diferencia de accuracy entre observado y esperado.

Parameters

df_comp -- DataFrame con columnas ['dataset','kernel','acc_diff']

`Analysis.plots.plot_decision_boundary(model: ClassifierMixin | Pipeline, X: ndarray, y: ndarray, ax: Axes, title: str = 'Decision Boundary') → None`

Plot decision boundary and support vectors for 2D data.

Parameters

- **model** -- Trained SVM or Pipeline containing an SVC with scaler
- **X** -- Array scaled or raw features (n_samples, 2)
- **y** -- True labels (n_samples,)
- **ax** -- Matplotlib Axes to plot on
- **title** -- Plot title

`Analysis.plots.plot_metrics_comparison(results, metrics=['accuracy', 'f1_score', 'n_support_vectors'], save_path=None)`

Plot bar charts comparing metrics per kernel. :param results: List of dicts per kernel for a given dataset :param metrics: Which metrics to plot :param save_path: If provided, path to save the figure as .png

1.3 Module contents

2.1 Submodules

2.2 Data.loader module

`Data.loader.load_all_datasets(uci_list: List[Tuple[int, str]], libsvm_dir: Path, libsvm_files: List[Tuple[str, str]]) → List[Tuple[str, ndarray, ndarray]]`

Carga datasets según la configuración recibida:

- `uci_list`: lista de (`dataset_id`, `nombre`)
- `libsvm_dir`: carpeta donde están los `.txt`
- `libsvm_files`: lista de (`filename`, `nombre`)

`Data.loader.load_expected_results()` → `DataFrame`

Devuelve un `DataFrame` con las métricas esperadas de precisión para cada par (`dataset`, `kernel`). Columnas: `['dataset', 'kernel', 'accuracy']`

2.3 Module contents

3.1 Submodules

3.2 Kernels.all_kernels module

`Kernels.all_kernels.build_kernels(hermite_degree: int = 6, gegen_degree: int = 3, gegen_alpha: float = 0.1) → List[Dict[str, Callable]]`

Construye lista de kernels con nombre y función.

3.3 Kernels.base module

`Kernels.base.linear() → Callable[[ndarray, ndarray], ndarray]`

Kernel lineal: $k(x, y) = x \cdot y$:return: función que computa la matriz de Gram

`Kernels.base.polynomial(degree: int = 3, gamma: float | None = None, coef0: float = 1.0) → Callable[[ndarray, ndarray], ndarray]`

Kernel polinómico: $(\gamma \langle x, y \rangle + \text{coef0})^{\text{degree}}$:param degree: grado del polinomio :param gamma: escala de la inner product (si None usa $1/n_{\text{features}}$) :param coef0: término independiente :return: función de kernel polinómico

`Kernels.base.rbf(gamma: float | None = None) → Callable[[ndarray, ndarray], ndarray]`

Kernel RBF (Gaussiano): $\exp(-\gamma \|x-y\|^2)$:param gamma: coeficiente (si None usa $1/n_{\text{features}}$) :return: función de kernel RBF

3.4 Kernels.custom module

`Kernels.custom.compute_gegenbauer(x: ndarray, n_max: int, alpha: float) → ndarray`

Genera polinomios de Gegenbauer $C_0..C_{n_{\text{max}}}(M)$ en cada x :param x: vector (n_{samples}) :param n_max: grado máximo :param alpha: parámetro :return: matriz ($n_{\text{samples}}, n_{\text{max}}+1$)

`Kernels.custom.compute_he(x: ndarray, max_order: int) → ndarray`

Computa polinomios de Hermite probabilistas $He_0 .. He_{max_order}$:param x: vector (n_samples,) :param max_order: grado máximo :return: matriz (n_samples, max_order+1)

`Kernels.custom.compute_u(i: int, alpha: float) → float`

Factor $u(C_i) = 1 / [\sqrt{i+1} * |C_i^{(1)}|]$

`Kernels.custom.gegenbauer_kernel(X: ndarray, Z: ndarray, n: int, alpha: float) → ndarray`

Kernel de Gegenbauer:

$K(x,z) = \sum_{j=1..d} [\sum_{i=0..n} C_i(x_j)C_i(z_j) * u_i^2 * w_{(x_j,z_j)}]$

`Kernels.custom.gegenbauer_weight(x: ndarray, z: ndarray, alpha: float, epsilon: float = 0.1) → ndarray`

Peso $w_{(x,z)} = [(1-x^2)(1-z^2)]^{(-1/2)} +$

`Kernels.custom.hermite_kernel_matrix(X: ndarray, Y: ndarray, n: int) → ndarray`

Kernel de Hermite probabilista vectorizado:

$K(x,z) = \sum_{j=0..d-1} [\sum_{i=0}^n 2^{-2i} He_i(x_j) He_i(z_j) * \exp(-(x_j^2+z_j^2)/2)]$

Parameters

- **X** -- array (m, d)
- **Y** -- array (p, d)
- **n** -- grado máximo de Hermite

Returns

matriz (m, p)

3.5 Module contents

4.1 Submodules

4.2 Model.evaluate module

`Model.evaluate.evaluate_model(model: ClassifierMixin, X_test: ndarray, y_test: ndarray) → Dict[str, float]`

Evalúa un modelo SVM entrenado en datos de test.

Parameters

- **model** -- Pipeline o SVC entrenado
- **X_test** -- Features de test escalados o pipeline que incluye scaler
- **y_test** -- Labels verdaderas

Returns

dict con 'accuracy', 'f1_score', 'n_support_vectors'

4.3 Model.train module

`Model.train.train_svm(X, y, kernel_func, C=1.0, cv=10, n_jobs=-1)`

Realiza validación cruzada con el kernel dado y devuelve métricas promedio, incluyendo estadísticas de vectores soporte.

`Model.train.tune_svm(X: ndarray, y: ndarray, kernel_func: Callable[[ndarray, ndarray], ndarray],
param_grid: Dict, cv: int = 5, n_jobs: int = -1) → BaseEstimator`

Realiza búsqueda de cuadrícula (GridSearchCV) para SVM con kernel custom.

Parameters

- **X** -- Features de entrenamiento
- **y** -- Labels
- **kernel_func** -- Kernel callable

- **param_grid** -- Diccionario de parámetros para GridSearchCV
- **cv** -- folds
- **n_jobs** -- cores

Returns

Mejor estimador

4.4 Module contents

CHAPTER 5

experiments module

```
experiments.main(config_path: str)
```


CHAPTER 6

hyper_opt module

`hyper_opt.generate_combinations(search_space)`

Para cada C en `search_space['svm.C']`, produce una combinación tomando en paralelo (por índice) los valores de los demás parámetros.

`hyper_opt.main()`

`hyper_opt.update_config(base_config, params)`

Actualiza la configuración con los nuevos parámetros.

a

Analysis, ??

Analysis.plots, ??

d

Data, ??

Data.loader, ??

e

experiments, ??

h

hyper_opt, ??

k

Kernels, ??

Kernels.all_kernels, ??

Kernels.base, ??

Kernels.custom, ??

m

Model, ??

Model.evaluate, ??

Model.train, ??